

1. Series

A **Series** is a one-dimensional labeled array capable of holding any data type (integers, strings, floats, Python objects, etc.). It's like a single column of a spreadsheet or a SQL table. The labels are called the **index**.

Key Characteristics:

- **Homogeneous data:** All elements in a Series are of the same data type.
- **Mutable size:** You can add or remove elements.
- **Labeled axis:** Data is accessed by labels (index), not just by integer position.

Examples:

Python

```
import pandas as pd
import numpy as np # Often used with Pandas
```

```
# --- Creating a Series ---
```

```
# 1. From a list (default integer index)
```

```
s1 = pd.Series([10, 20, 30, 40, 50])
```

```
print("Series from a list:\n", s1)
```

```
# 2. From a dictionary (keys become the index)
```

```
data_dict = {'apple': 3, 'banana': 5, 'cherry': 7}
```

```
s2 = pd.Series(data_dict)
```

```
print("\nSeries from a dictionary:\n", s2)
```

```
# 3. With a custom index
```

```
s3 = pd.Series([100, 200, 300], index=['a', 'b', 'c'])
```

```
print("\nSeries with custom index:\n", s3)
```

```
# 4. From a scalar value (repeats the value for the given index)
```

```
s4 = pd.Series(5, index=['x', 'y', 'z'])
```

```

print("\nSeries from a scalar:\n", s4)

# --- Series Attributes ---
print("\n--- Series Attributes ---")
print("Index of s3:", s3.index)
print("Values of s3:", s3.values)
print("Data type of s3:", s3.dtype)
print("Shape of s3:", s3.shape) # Returns a tuple (number_of_elements,)
print("Number of dimensions of s3:", s3.ndim) # Always 1 for a Series

# --- Series Operations ---
print("\n--- Series Operations ---")
s_ops = pd.Series([10, 20, 30, 40, 50], index=['A', 'B', 'C', 'D', 'E'])

# Accessing elements
print("Element at index 'B':", s_ops['B'])
print("Element at position 0:", s_ops[0])
print("Elements from 'B' to 'D' (inclusive for label-based):\n",
s_ops['B':'D'])
print("Elements from position 1 to 3 (exclusive for position-based):\n",
s_ops[1:4])

# Conditional selection (Boolean indexing)
print("\nElements greater than 30:\n", s_ops[s_ops > 30])

# Arithmetic operations (element-wise)
print("\nSeries + 5:\n", s_ops + 5)
print("Series * 2:\n", s_ops * 2)

# Mathematical functions
print("\nSquare root of series:\n", np.sqrt(s_ops))

# Aggregation methods
print("Mean of series:", s_ops.mean())
print("Sum of series:", s_ops.sum())
print("Max of series:", s_ops.max())

```

2. DataFrame

A `DataFrame` is a two-dimensional labeled data structure with columns of potentially

different types. It's the most commonly used Pandas object, resembling a spreadsheet or a SQL table. You can think of it as a dictionary of `Series` objects where all `Series` share the same index.

Key Characteristics:

- **Tabular data:** Organized into rows and columns.
- **Heterogeneous data:** Columns can have different data types.
- **Mutable size and mutable values:** You can add/remove columns/rows and modify values.
- **Labeled axes:** Both rows (index) and columns have labels.

Examples:

Python

```
import pandas as pd
import numpy as np
```

```
# --- Creating a DataFrame ---
```

```
# 1. From a dictionary of lists (common way)
```

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston'],
    'Salary': [70000, 80000, 90000, 95000]
}
df = pd.DataFrame(data)
print("DataFrame from a dictionary of lists:\n", df)
```

```
# 2. From a list of dictionaries
```

```
data_list_of_dicts = [
    {'Name': 'Eve', 'Age': 22, 'City': 'Miami', 'Salary': 65000},
    {'Name': 'Frank', 'Age': 28, 'City': 'Boston', 'Salary': 75000}
]
df_list = pd.DataFrame(data_list_of_dicts)
print("\nDataFrame from a list of dictionaries:\n", df_list)
```

```
# 3. From a NumPy array (requires specifying columns and/or index)
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
df_arr = pd.DataFrame(arr, columns=['ColA', 'ColB', 'ColC'],
index=['Row1', 'Row2', 'Row3'])
print("\nDataFrame from a NumPy array:\n", df_arr)

# --- DataFrame Attributes ---
print("\n--- DataFrame Attributes ---")
print("Index:", df.index)
print("Columns:", df.columns)
print("Values (as NumPy array):\n", df.values)
print("Shape (rows, columns):", df.shape)
print("Data types:\n", df.dtypes)
```

3. Basic DataFrame Operations & Inspection

Python

```
import pandas as pd
import numpy as np

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
    'Age': [25, 30, 35, 40, 22, 28],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Miami',
'Boston'],
    'Salary': [70000, 80000, 90000, 95000, 65000, 75000]
}
df = pd.DataFrame(data)

print("Original DataFrame:\n", df)

# Display first/last few rows
print("\nFirst 3 rows (head):\n", df.head(3))
print("\nLast 2 rows (tail):\n", df.tail(2))
```

```

# Get concise summary of the DataFrame
print("\nDataFrame Info:")
df.info() # Shows data types, non-null counts, memory usage

# Get descriptive statistics for numerical columns
print("\nDescriptive Statistics:\n", df.describe())

# Get a specific column (returns a Series)
print("\n'Name' column:\n", df['Name'])
print("\n'Age' column (alternative syntax for single-word columns):\n",
df.Age)

# Get multiple columns (returns a DataFrame)
print("\n'Name' and 'Salary' columns:\n", df[['Name', 'Salary']])

# Adding a new column
df['Experience'] = [3, 7, 10, 15, 2, 6]
print("\nDataFrame after adding 'Experience' column:\n", df)

# Modifying an existing column
df['Salary'] = df['Salary'] * 1.05 # 5% raise
print("\nDataFrame after 5% salary raise:\n", df)

# Deleting a column
df_no_experience = df.drop(columns=['Experience'])
print("\nDataFrame after dropping 'Experience' column (original df
unchanged):\n", df_no_experience)

# To drop in-place (modifies the original DataFrame)
df.drop(columns=['City'], inplace=True)
print("\nDataFrame after dropping 'City' column (in-place):\n", df)

```

4. Selection and Indexing (.loc and .iloc)

These are crucial for selecting data based on labels or integer positions.

- `.loc[]`: Label-based indexing. Used with row/column labels.
- `.iloc[]`: Integer-location based indexing. Used with integer positions (like NumPy arrays).

Python

```
import pandas as pd
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],  
    'Age': [25, 30, 35, 40, 22],  
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Miami'],  
    'Salary': [70000, 80000, 90000, 95000, 65000]  
}  
df = pd.DataFrame(data, index=['A', 'B', 'C', 'D', 'E'])  
print("Original DataFrame with custom index:\n", df)
```

```
# --- Using .loc[] (Label-based) ---  
print("\n--- Using .loc[] (Label-based) ---")
```

```
# Select a single row by label  
print("Row with label 'C':\n", df.loc['C'])
```

```
# Select multiple rows by label  
print("\nRows with labels 'A' and 'D':\n", df.loc[['A', 'D']])
```

```
# Select rows and columns by labels  
print("\nName' and 'Salary' for rows 'A' to 'C' (inclusive of 'C'):\n",  
df.loc['A':'C', ['Name', 'Salary']])
```

```
# Conditional selection with .loc  
print("\nRows where Age > 30 using .loc:\n", df.loc[df['Age'] > 30])
```

```
# Modify values using .loc  
df.loc['A', 'Salary'] = 75000  
print("\nDataFrame after modifying 'Alice's Salary using .loc:\n", df)
```

```
# --- Using .iloc[] (Integer-location based) ---  
print("\n--- Using .iloc[] (Integer-location based) ---")
```

```
# Select a single row by position  
print("Row at position 1 (2nd row):\n", df.iloc[1])
```

```
# Select multiple rows by position
print("\nRows at positions 0 and 3:\n", df.iloc[[0, 3]])

# Select rows and columns by positions
print("\nRows at positions 0-2 and columns 0-1:\n", df.iloc[0:3, 0:2]) #
Rows 0,1,2 and columns 0,1 (exclusive of end)

# Modify values using .iloc
df.iloc[0, 1] = 26 # Modify Age of first person (position 0, column 1)
print("\nDataFrame after modifying Age of first person using .iloc:\n", df)
```

5. Handling Missing Data

Real-world data often contains missing values (represented as `NaN` - Not a Number). Pandas provides robust tools to handle them.

Python

```
import pandas as pd
import numpy as np

df_missing = pd.DataFrame({
    'A': [1, 2, np.nan, 4, 5],
    'B': [np.nan, 6, 7, 8, 9],
    'C': [10, 11, 12, np.nan, 14],
    'D': [15, 16, 17, 18, np.nan]
})
print("DataFrame with missing values:\n", df_missing)

# Check for missing values
print("\nIs null (Boolean DataFrame):\n", df_missing.isnull())
print("\nIs not null:\n", df_missing.notnull())

# Count missing values per column
print("\nMissing values per column:\n", df_missing.isnull().sum())
```

```
print("Total missing values in DataFrame:",  
df_missing.isnull().sum().sum())
```

```
# --- Strategies for Handling Missing Data ---
```

```
# 1. Dropping missing values
```

```
# Drop rows with ANY missing values
```

```
df_cleaned_rows = df_missing.dropna()
```

```
print("\nDataFrame after dropping rows with any NaN:\n",  
df_cleaned_rows)
```

```
# Drop rows if ALL values are missing (not applicable here, but useful)
```

```
df_all_nan_row = pd.DataFrame({'X': [1, np.nan], 'Y': [2, np.nan]})
```

```
df_all_nan_row.loc[2] = [np.nan, np.nan]
```

```
print("\nOriginal with all NaN row:\n", df_all_nan_row)
```

```
print("After dropping rows where ALL are NaN:\n",
```

```
df_all_nan_row.dropna(how='all'))
```

```
# Drop columns with ANY missing values
```

```
df_cleaned_cols = df_missing.dropna(axis=1) # axis=1 or columns
```

```
print("\nDataFrame after dropping columns with any NaN:\n",  
df_cleaned_cols)
```

```
# 2. Filling missing values (Imputation)
```

```
# Fill with a scalar value (e.g., 0)
```

```
df_filled_zero = df_missing.fillna(0)
```

```
print("\nDataFrame after filling NaN with 0:\n", df_filled_zero)
```

```
# Fill with the mean of the column
```

```
df_filled_mean = df_missing.fillna(df_missing.mean())
```

```
print("\nDataFrame after filling NaN with column mean:\n",  
df_filled_mean)
```

```
# Fill with previous valid observation (Forward fill - ffill)
```

```
df_ffill = df_missing.ffill()
```

```
print("\nDataFrame after forward fill (ffill):\n", df_ffill)
```

```
# Fill with next valid observation (Backward fill - bfill)
```

```
df_bfill = df_missing.bfill()
```

```
print("\nDataFrame after backward fill (bfill):\n", df_bfill)
```

```
# Fill with specific values for different columns
df_filled_specific = df_missing.fillna({'A': df_missing['A'].mean(), 'B': 0})
print("\nDataFrame after filling 'A' with mean and 'B' with 0:\n",
df_filled_specific)
```

6. Grouping and Aggregation (groupby())

The `groupby()` method is one of the most powerful features, allowing you to split data into groups based on some criteria, apply a function to each group independently, and then combine the results.

Python

```
import pandas as pd

data = {
    'Department': ['HR', 'IT', 'HR', 'IT', 'Marketing', 'IT', 'Marketing', 'HR'],
    'Gender': ['F', 'M', 'F', 'M', 'F', 'F', 'M', 'M'],
    'Salary': [60000, 90000, 65000, 95000, 70000, 88000, 72000, 70000],
    'Years_Exp': [3, 7, 4, 8, 5, 6, 4, 5]
}
df_agg = pd.DataFrame(data)
print("Original DataFrame for aggregation:\n", df_agg)

# Group by 'Department' and calculate mean salary
print("\nMean Salary by Department:\n", df_agg.groupby('Department')
['Salary'].mean())

# Group by 'Department' and 'Gender' and calculate count
print("\nCount by Department and Gender:\n",
df_agg.groupby(['Department',
'Gender']).size().reset_index(name='Count'))

# Multiple aggregations using .agg()
agg_results = df_agg.groupby('Department').agg(
```

```

    Avg_Salary=('Salary', 'mean'),
    Min_Salary=('Salary', 'min'),
    Max_Salary=('Salary', 'max'),
    Num_Employees=('Salary', 'count'),
    Total_Exp=('Years_Exp', 'sum')
)
print("\nMultiple aggregations by Department:\n", agg_results)

# Apply a custom aggregation function
def salary_range(series):
    return series.max() - series.min()

range_results = df_agg.groupby('Department')
['Salary'].agg(salary_range)
print("\nSalary Range by Department:\n", range_results)

```

7. Merging, Joining, and Concatenating

These operations are used to combine DataFrames.

- **pd.merge():** Used for combining DataFrames based on common columns or indices, similar to SQL JOIN operations.
 - **how='inner'** (default): Returns only rows with matching keys in both DataFrames.
 - **how='outer'**: Returns all rows when there is a match in one of the DataFrames.
 - **how='left'**: Returns all rows from the left DataFrame and matching rows from the right.
 - **how='right'**: Returns all rows from the right DataFrame and matching rows from the left.
- **df.join():** A convenient method for merging DataFrames on their indexes or on a key column.
- **pd.concat():** Stacks DataFrames either vertically (rows) or horizontally (columns).

Python

```

import pandas as pd
import numpy as np

df1 = pd.DataFrame({'key': ['A', 'B', 'C', 'D'], 'value': np.random.randint(1,
10, 4)})
df2 = pd.DataFrame({'key': ['B', 'D', 'E', 'F'], 'value2':
np.random.randint(10, 20, 4)})
print("DataFrame 1:\n", df1)
print("\nDataFrame 2:\n", df2)

# --- Merging (pd.merge) ---
print("\n--- Merging (pd.merge) ---")

# Inner Merge
merged_inner = pd.merge(df1, df2, on='key', how='inner')
print("\nInner Merge:\n", merged_inner)

# Outer Merge
merged_outer = pd.merge(df1, df2, on='key', how='outer')
print("\nOuter Merge:\n", merged_outer)

# Left Merge
merged_left = pd.merge(df1, df2, on='key', how='left')
print("\nLeft Merge:\n", merged_left)

# Right Merge
merged_right = pd.merge(df1, df2, on='key', how='right')
print("\nRight Merge:\n", merged_right)

# Merging on multiple keys
df3 = pd.DataFrame({'key1': ['A', 'B', 'C'], 'key2': ['X', 'Y', 'Z'], 'value3': [1,
2, 3]})
df4 = pd.DataFrame({'key1': ['A', 'B', 'C'], 'key2': ['X', 'Z', 'Y'], 'value4': [4,
5, 6]})
merged_multi_key = pd.merge(df3, df4, on=['key1', 'key2'], how='inner')
print("\nMerge on multiple keys:\n", merged_multi_key)

# --- Concatenation (pd.concat) ---
print("\n--- Concatenation (pd.concat) ---")

df_concat1 = pd.DataFrame({'A': ['A0', 'A1'], 'B': ['B0', 'B1']}, index=[0, 1])

```

```
df_concat2 = pd.DataFrame({'A': ['A2', 'A3'], 'B': ['B2', 'B3']}, index=[2, 3])
df_concat3 = pd.DataFrame({'C': ['C4', 'C5'], 'D': ['D4', 'D5']}, index=[0,
1])
```

```
print("\nDataFrame Concat 1:\n", df_concat1)
print("\nDataFrame Concat 2:\n", df_concat2)
print("\nDataFrame Concat 3:\n", df_concat3)
```

```
# Concatenate along rows (default, axis=0)
concat_rows = pd.concat([df_concat1, df_concat2])
print("\nConcatenation along rows:\n", concat_rows)
```

```
# Concatenate along columns (axis=1)
concat_cols = pd.concat([df_concat1, df_concat3], axis=1)
print("\nConcatenation along columns:\n", concat_cols)
```

```
# Concatenate with ignore_index=True to reset index
concat_reset_index = pd.concat([df_concat1, df_concat2],
ignore_index=True)
print("\nConcatenation along rows with reset index:\n",
concat_reset_index)
```

8. Applying Functions

Pandas provides various methods to apply functions to Series or DataFrames.

- `.apply()`: Can be used on a Series to apply a function element-wise, or on a DataFrame to apply a function along rows or columns.
- `.map()`: Used on a Series for element-wise mapping, typically with a dictionary or Series.
- `.applymap()`: Used on a DataFrame to apply a function element-wise across all elements (less common now, `df.map()` or direct vectorized operations are often preferred).

Python

```
import pandas as pd
```

```

df_apply = pd.DataFrame({
    'Value1': [10, 20, 30, 40],
    'Value2': [100, 200, 300, 400],
    'Category': ['A', 'B', 'A', 'C']
})
print("Original DataFrame for apply:\n", df_apply)

# --- Using .apply() ---

# Apply a lambda function to a column (Series) - element-wise
df_apply['Value1_squared'] = df_apply['Value1'].apply(lambda x: x**2)
print("\nDataFrame with 'Value1_squared' (using .apply on Series):\n",
df_apply)

# Apply a function to each row (axis=1) of a DataFrame
def calculate_product(row):
    return row['Value1'] * row['Value2']

df_apply['Product'] = df_apply.apply(calculate_product, axis=1)
print("\nDataFrame with 'Product' (using .apply on DataFrame rows):\n",
df_apply)

# Apply a function to each column (axis=0) of a DataFrame (less
common for simple aggregations)
# Example: Get the sum of each numerical column
column_sums = df_apply[['Value1', 'Value2']].apply(sum, axis=0)
print("\nSum of numerical columns (using .apply on DataFrame
columns):\n", column_sums)

# --- Using .map() ---
# Used for substituting each value in a Series with another value.
# Useful for categorical data mapping.
category_mapping = {'A': 'High', 'B': 'Medium', 'C': 'Low'}
df_apply['Category_Text'] =
df_apply['Category'].map(category_mapping)
print("\nDataFrame with 'Category_Text' (using .map on Series):\n",
df_apply)

# --- Using vectorized operations (preferred when possible) ---
# For simple element-wise operations, direct vectorized operations are

```

```
much faster
df_apply['Value1_plus_Value2'] = df_apply['Value1'] + df_apply['Value2']
print("\nDataFrame with 'Value1_plus_Value2' (using vectorized
operation):\n", df_apply)
```

9. Reading and Writing Data

Pandas makes it incredibly easy to read data from and write data to various file formats.

Python

```
import pandas as pd
import os

# Let's use our 'df' from previous examples
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df = pd.DataFrame(data)

# --- Writing Data ---

# To CSV
csv_file = 'my_data.csv'
df.to_csv(csv_file, index=False) # index=False prevents writing the
DataFrame index as a column
print(f"\nDataFrame saved to '{csv_file}'")

# To Excel (requires openpyxl or xlswriter: pip install openpyxl)
excel_file = 'my_data.xlsx'
try:
    df.to_excel(excel_file, index=False, sheet_name='Sheet1')
    print(f"DataFrame saved to '{excel_file}'")
```

```

except ImportError:
    print(f"To write to Excel, install openpyxl: pip install openpyxl")
except Exception as e:
    print(f"Error saving to Excel: {e}")

# To JSON
json_file = 'my_data.json'
df.to_json(json_file, orient='records', indent=4) # orient controls JSON
format, indent for readability
print(f"DataFrame saved to '{json_file}'")

# --- Reading Data ---

# From CSV
try:
    df_read_csv = pd.read_csv(csv_file)
    print(f"\nDataFrame read from '{csv_file}':\n", df_read_csv)
except FileNotFoundError:
    print(f"\n'{csv_file}' not found. Please ensure it was created
successfully.")

# From Excel
try:
    df_read_excel = pd.read_excel(excel_file, sheet_name='Sheet1')
    print(f"\nDataFrame read from '{excel_file}':\n", df_read_excel)
except FileNotFoundError:
    print(f"\n'{excel_file}' not found. Please ensure it was created
successfully.")
except Exception as e:
    print(f"\nError reading Excel file: {e}")

# From JSON
try:
    df_read_json = pd.read_json(json_file)
    print(f"\nDataFrame read from '{json_file}':\n", df_read_json)
except FileNotFoundError:
    print(f"\n'{json_file}' not found. Please ensure it was created
successfully.")

# Cleanup (optional)

```

```
if os.path.exists(csv_file):  
    os.remove(csv_file)  
if os.path.exists(excel_file):  
    os.remove(excel_file)  
if os.path.exists(json_file):  
    os.remove(json_file)  
print("\nCleaned up generated files.")
```